

中山大学计算机学院编译器构造实验报告

姓名	学号	实验序号
范益嘉	23336064	实验 1：词法分析

一、实验过程

1.1 实验目标概述

本次实验的目标是基于 Flex 工具实现一个 SysY 语言的词法分析器。词法分析器需要能够：

- 识别 SysY 语言的所有关键字、标识符、常量、运算符和分隔符；
- 正确解析 C 预处理器输出的行号/文件名指令 (`# lineum "filename" flags`)；
- 追踪每个 token 在源文件中的精确位置 (文件名: 行号: 列号)；
- 正确标记 `[StartOfLine]` 和 `[LeadingSpace]` 属性；
- 输出格式与 `clang -cc1 -dump-tokens` 保持一致。

1.2 实验框架分析

实验提供了基于 Flex 的框架代码，包含以下核心文件：

- `lex.l`：Flex 规则文件，定义词法规则，是主要的编码工作所在；
- `lex.hpp`：头文件，定义 `enum Id` (token 类型枚举) 和全局状态结构体 `G`；
- `lex.cpp`：实现 `come()` 函数 (处理识别到的 token) 和 `id2str()` 函数 (token 类型到字符串的映射)；
- `main.cpp`：主程序，包含 `print_token()` 输出函数和 `main()` 入口。

框架已经提供了部分 token 的识别规则 (如 `int`、`return`、括号、`+`、`;` 等)，但缺少大量关键字、运算符，且未实现预处理指令解析和位置追踪。

1.3 需求分析——确定完整的 token 集合

通过分析测试用例和 `clang -cc1 -dump-tokens` 的标准答案输出，我确定了 SysY 语言需要支持的完整 token 集合：

- 关键字 (9 个): `int`, `return`, `const`, `void`, `if`, `else`, `while`, `break`, `continue`
- 标识符和字面量: `identifier`, `numeric_constant` (十进制、八进制、十六进制), `string_literal`
- 分隔符 (8 个): `(`, `)`, `[`, `]`, `{`, `}`, `;`, `,`
- 运算符 (15 个): `=`, `+`, `-`, `*`, `/`, `%`, `<`, `>`, `<=`, `>=`, `==`, `!=`, `&&`, `||`, `!`

关键发现：通过运行以下命令提取标准答案中的全部 token 类型：

```
1 find build/test/task1 -name "answer.txt" \
2 -exec cat {} \; | awk '{print $1}' | sort -u
```

能快速确定需要支持的所有 token 名称，避免遗漏。

1.4 关键实现细节

(1) Flex 词法规则的编写 (lex.l)

核心的 COME 宏被重新设计以同时完成列号追踪和 token 报告：

```
1 #define COME(id) do { \
2     int _startCol = g.mColumn; \
3     g.mLine = yylineno; \
4     g.mColumn += yyleng; \
5     return come(id, yytext, yyleng, _startCol); \
6 } while(0)
```

这个宏的关键在于：先保存当前列号作为 token 的起始列 (_startCol)，再将列号更新到 token 结束之后的位置 (g.mColumn += yyleng)，最后调用 come() 时传入起始列号。

多字符运算符必须在单字符运算符之前定义，利用 Flex 的最长匹配原则：

```
1 "<="      { COME(LESSEQUAL); }
2 ">="      { COME(GREATEREQUAL); }
3 "=="      { COME(EQUALEQUAL); }
4 "!="      { COME(EXCLAMEQUAL); }
5 "&&"      { COME(AMPAMP); }
6 "||"      { COME(PIPEPIPE); }
7 "<"       { COME(LESS); } /* 单字符在后 */
8 ">"       { COME(GREATER); }
```

十六进制常量的匹配规则是框架中缺少的重要部分：

```
1 H      [a-fA-F0-9]
2 0[xX]{H}+{IS}? { COME(CONSTANT); }
```

(2) 预处理指令解析

预处理后的文件包含形如 # 1 "filename" 2 的行号指令。解析逻辑如下：

```
1 ^#[^\n]* {
2     const char* p = yytext + 1; /* 跳过 '#' */
3     while (*p == ' ') p++;
4     int line = 0;
5     while (*p >= '0' && *p <= '9') {
6         line = line * 10 + (*p - '0');
7         p++;
8     }
9     while (*p == ' ') p++;
10    if (*p == '"') {
11        p++;
```

```

12  const char* start = p;
13  while (*p && *p != '\n') p++;
14  g.mFile = std::string(start, p - start);
15  }
16  yylineno = line - 1; /* 减1因为后续\n会+1 */
17  g.mColumn = 1;
18  g.mStartOfLine = true;
19  return ~YYEOF;
20 }

```

设置 `yylineno = line - 1` 是因为此规则不消费行末的 `\n`，当 Flex 随后匹配 `\n` 时会自动将 `yylineno` 加 1，使得下一行代码的行号恰好正确。

(3) 空白字符与位置追踪

空白字符的处理对正确生成 `[StartOfLine]` 和 `[LeadingSpace]` 标记至关重要：

```

1  [ \t\v\f] { /* 水平空白 */
2    g.mColumn += yyleng;
3    g.mLeadingSpace = true;
4    return ~YYEOF;
5  }
6  \n { /* 换行符 */
7    g.mColumn = 1;
8    g.mStartOfLine = true;
9    g.mLeadingSpace = false;
10   return ~YYEOF;
11 }

```

(4) EOF 位置的特殊处理

clang 将 EOF token 的位置报告为最后一个真实 token 之后的位置，而不是文件末尾换行符之后。为此我在 struct G 中增加了两个字段追踪最后一个 token 的结束位置：

```

1 // lex.hpp 中的 struct G
2 int mLastTokenEndCol{ 1 }; // 上一个 token 结束列
3 int mLastTokenEndLine{ 1 }; // 上一个 token 结束行

```

在 `come()` 函数中对 EOF 做特殊处理：

```

1 if (g.mId == Id::YYEOF) {
2   g.mColumn = g.mLastTokenEndCol;
3   g.mLine = g.mLastTokenEndLine;
4   g.mStartOfLine = false;
5   g.mLeadingSpace = false;
6   print_token();
7 } else {
8   // ... 正常 token 处理 ...
9   g.mLastTokenEndCol = endCol;
10  g.mLastTokenEndLine = g.mLine;
11 }

```

(5) 输出格式修正 (main.cpp)

标准答案的输出格式需要精确匹配，包括制表符和空格的位置：

```
1 // token_name 'value'\t [Flag1] [Flag2]\tLoc=<file:line:col>
2 if (lex::g.mStartOfLine || lex::g.mLeadingSpace) {
3     outFile << "\t";
4     if (lex::g.mStartOfLine)
5         outFile << " [StartOfLine]";
6     if (lex::g.mLeadingSpace)
7         outFile << " [LeadingSpace]";
8 } else {
9     outFile << "\t";
10 }
11 outFile << "\tLoc=<" << lex::g.mFile << ":"
12     << lex::g.mLine << ":" << lex::g.mColumn << ">";
```

当两个标志同时存在时输出 \t [StartOfLine] [LeadingSpace]\tLoc=..., 无标志时输出两个连续制表符 \t\tLoc=...。

1.5 实验技巧总结

1. 善用 `clang -cc1 -dump-tokens` 作为标准参考, 分析其输出的精确格式;
2. 使用 `cat -A` 查看文件中不可见字符 (制表符 `^I`、行末 `$`), 确保输出格式完全匹配;
3. 先实现基本功能通过简单测例, 再逐步扩展到完整 token 集合;
4. Flex 的 `%option yylineno` 自动追踪行号, 但需要理解其与手动覆盖的交互方式。

1.6 实验结果

最终实验评测结果: 总分 100.00/100.00, 所有 68 个测例全部通过, 包括 functional-0 (11 例)、functional-1 (15 例)、functional-2 (6 例)、functional-3 (27 例) 和 mini-performance (9 例) 五个测试集。

二、AI 使用

2.1 模型与 token 用量

- 使用模型: Claude Opus 4.6 (1M context)
- Token 使用量: 约 100K+ tokens (包含代码阅读、生成、编译调试全流程)

2.2 AI 使用心得

(1) Prompt 方面

最有效的 prompt 策略是给出高层目标, 让 AI 自主探索。本次实验中直接说明”帮我完成 task1 并提交评测”, AI 能够自动完成以下 workflow:

- 阅读项目 README 理解整体框架;
- 分析 task1 目录下所有源文件理解现有代码结构;
- 查看测试用例和评分脚本理解需求和评判标准;

- 从标准答案反推需要支持的完整 token 集合；
- 编写代码、编译、测试、修复，形成完整的开发闭环。

(2) Coding 方面

AI 在以下几个方面表现出色：

- 全局代码理解：能够同时理解 4 个相互关联的文件 (lex.l、lex.hpp、lex.cpp、main.cpp)，确保枚举值、字符串数组和规则三者的一致性；
- 快速调试迭代：遇到列号偏移的 bug 时，AI 能够通过添加调试输出、分析宏展开逻辑、定位到 ADDCOL() 被重复调用的根因，并在数分钟内修复；
- 边界情况处理：自动识别并处理了 EOF 位置、预处理指令行号同步等容易遗漏的细节。

(3) Chat 方面

在与 AI 的交互中，保持任务导向（“完成实验”而非“写某个函数”）能让 AI 发挥更大的自主性，减少人工干预。同时，AI 能够在遇到问题时自动查阅项目中的相关文件寻找答案，而非仅依赖已有知识。

三、实验建议

3.1 实验内容建议

1. 明确 token 列表：建议在 task1 的 README 中提供完整的 SysY 语言 token 类型列表。当前需要通过运行 clang 或分析测试用例来推断，增加了不必要的探索成本。
2. 预处理指令文档：预处理指令的解析（提取文件名和行号）是隐含需求，建议在文档中明确说明其格式和解析要求。
3. 输出格式规范：建议提供输出格式的精确说明（如制表符和空格的使用），而非仅通过示例展示。当前需要使用 cat -A 分析标准答案才能确定格式细节。
4. 评分脚本：评分脚本设计合理，三级评分（token 类型 60%、位置 30%、无关字符标记 10%）能够有效区分不同完成度。

3.2 AI 服务建议

1. AI 辅助完成编译实验整体效果良好，尤其在框架理解和快速迭代方面优势明显。
2. 建议提供一份“如何有效利用 AI 辅助完成编译实验”的指南文档，帮助学生了解合理的使用方式。
3. 在 Agent 模式下，AI 能够自主完成“读代码—写代码—编译—测试—修 bug”的完整循环，这种工作流特别适合编译实验这类有明确输入输出规范的任务。